

Laboratorio 2

Diego Stefanini — prof. Patrizio Dazzi, Luca Ferrucci, a.a. 2026-27

Indice

1	L'ambiente di lavoro UNIX	3
1.1	Il terminale e il filesystem	3
1.2	Lavorare con i file	4
1.3	Flussi: redirectione e pipe	6
2	Il linguaggio C	8
2.1	Dal sorgente all'eseguibile	8
2.2	Com'è fatto un programma C	10
2.3	Tipi e costanti	12
2.4	Operatori ed espressioni	14
2.5	Il controllo del flusso	16

1 L'ambiente di lavoro UNIX

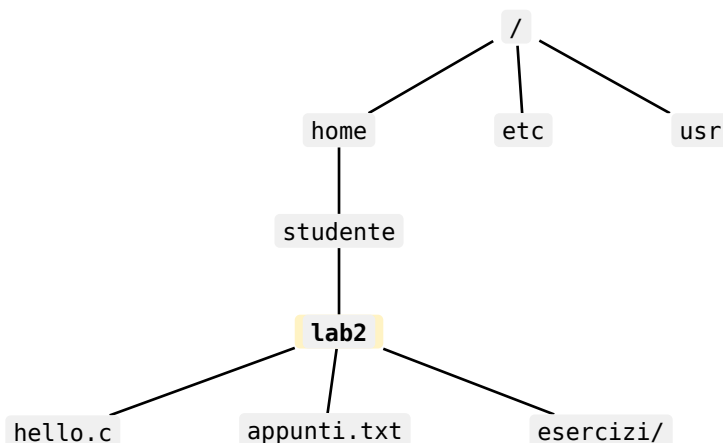
1.1 Il terminale e il filesystem

Si lavora dal **terminale**: una finestra in cui gira la **shell**, il programma che legge i comandi e li esegue.

```
$ pwd
/home/studente/lab2
$ ls
hello.c  appunti.txt
```

- **Terminale**: interfaccia testuale per dialogare con il sistema.
- **Shell** (bash, zsh): il programma dentro il terminale che interpreta i comandi.
- Il **prompt** (\$) dice che la shell aspetta un comando. Non fa parte del comando e non è sempre \$.

I comandi agiscono sul **filesystem**, un albero di directory e file:



Il filesystem è un **albero**. Parte da `/`, la **root**: ogni file o directory si raggiunge con un **percorso**. `/` separa un livello dal successivo.

File e directory sono oggetti diversi ma stanno nello stesso spazio dei nomi.

In giallo la **directory corrente**.

- `.` = la directory corrente
- `..` = la directory padre

Comando	Descrizione
<code>pwd</code>	<i>print working directory</i> : stampa in quale cartella sei
<code>ls</code>	elenca il contenuto della directory corrente
<code>ls -l</code>	elenco in formato lungo (permessi, proprietario, dimensione, data)
<code>cd</code>	<i>change directory</i> : navigo in una directory
<code>mkdir</code>	<i>make directory</i> : crea una directory
<code>echo \$?</code>	restituisce il valore di ritorno dell'ultimo comando eseguito

Per indicare un file nell'albero si usa un **percorso**, assoluto o relativo:

Relativo: parte dalla directory corrente (es. `esercizi/es1.c`). **Assoluto**: parte dalla root `/` (es. `/home/studente/lab2`).

```
$ cd esercizi
$ pwd
/home/studente/lab2/esercizi
```

```
$ cd /home/studente
$ pwd
/home/studente
```

```
$ pwd
/home/studente/lab2
$ cd ..      # vado nel padre
$ pwd
/home/studente
```

Due casi speciali: la **home** e i **file nascosti**.

`~` = la **home** dell'utente corrente. `cd` senza argomenti porta di solito lì.

```
$ cd ~
$ pwd
/home/studente
```

Un nome che inizia con `.` è **nascosto**: `ls` non lo mostra, `ls -a` sì.

```
$ ls
appunti.txt hello.c
$ ls -a
. .. .bashrc appunti.txt hello.c
```

«Nascosto» è solo una **convenzione sul nome**, non un tipo speciale di file.

Con `ls -l` si vedono le informazioni su ogni file. Ecco come leggerle:

<code>d</code>	<code>rwxr-xr-x</code>	<code>studente</code>	<code>studenti</code>	<code>64</code>	<code>Sep 15 09:10</code>	<code>esercizi</code>
tipo: d = directory, - = file	permessi	proprietario	gruppo	dimensione (byte)	ultima modifica	nome

Ogni comando ha la stessa forma. Per esempio `gcc`, il compilatore C che si usa per tutto il corso:

<code>gcc</code>	<code>-Wall</code>	<code>hello.c</code>	<code>-o hello</code>
comando	opzione	argomento	opzione + argomento

Le **opzioni** modificano il comportamento del comando. Se non ricordi come si usa: `gcc --help` (sintesi rapida) oppure `man gcc` (manuale UNIX).

Gli argomenti li prepara la shell: **spezza la riga in parole usando gli spazi** e avvia il programma passandogli gli argomenti. Se un nome contiene spazi, le **virgolette** lo tengono insieme come un solo argomento:

```
$ touch "appunti lezione.txt" # un file, non due
$ cat "appunti lezione.txt"
```

Vedremo come un programma C legge questi argomenti con `argc` e `argv`, i parametri di `main`.

1.2 Lavorare con i file

Per creare directory e file vuoti si usano `mkdir` e `touch`:

```
$ mkdir esercizi # argomento = nome
$ touch appunti.txt # file vuoto
```

`touch` in realtà nasce per **cambiare i timestamp** di un file. Se il file non esiste lo crea: la creazione è una conseguenza utile, non lo scopo.

Oltre al contenuto, per ogni file UNIX tiene tre marcature temporali (**timestamp**). `stat file` le mostra, insieme agli altri metadati.

Metadato	Significato
<code>atime</code>	<i>access time</i> : ultimo accesso al contenuto
<code>mtime</code>	<i>modification time</i> : ultima modifica del contenuto
<code>ctime</code>	<i>change time</i> : ultima modifica dei metadati

`ctime` vuol dire **change** time, **non** «creation time». Alcuni filesystem moderni salvano anche il tempo di creazione (*birth time*), ma non fa parte dei tre timestamp UNIX classici.

A cosa servono: backup incrementali, sincronizzazione, diagnostica, cercare i file modificati di recente, e ricompilare solo ciò che è cambiato. `make` (lo vedremo più avanti) confronta i timestamp per decidere se ricompilare.

Chi aggiorna cosa (✓ = portato all'ora corrente):

Operazione	<code>atime</code>	<code>mtime</code>	<code>ctime</code>
<code>touch file</code>	✓	✓	✓ (cambiare i tempi è già una modifica dei metadati)
<code>touch -a file</code>	✓	—	✓
<code>touch -m file</code>	—	✓	✓
<code>cp</code> : destinazione	✓ solo se creato	✓	✓
<code>cp -p</code> : destinazione	copiato dal sorgente	copiato dal sorgente	✓
<code>cp</code> : sorgente	✓ può, per la lettura	—	—

Per copiare, spostare e cancellare:

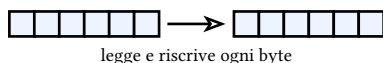
`cp` sorgente destinazione (*copy*): l'originale resta al suo posto.

```
$ cp hello.c copia.c
$ cp /tmp/ciao.txt ~/hello.txt # copia
e rinomina
```

`mv` (*move*): un comando, **due usi**.

```
$ mv copia.c esempio.c # rinomina
$ mv esempio.c esercizi/ # sposta
```

copia — $O(N)$



sposta (stesso filesystem) — $O(1)$

cambia solo il **percorso** nei metadati

Operazione	Costo
copia	$O(N)$
sposta, stesso filesystem	$O(1)$
sposta, altro filesystem	$O(N)$: copia + cancella

Comando	Descrizione
<code>rm</code>	cancella un file (<i>remove</i>)
<code>rmdir</code>	cancella una directory vuota
<code>rm -r</code>	cancella una directory con tutto il contenuto. <code>-r</code> = ricorsivamente : attraversa tutto ciò che c'è dentro

`rm` **non** sposta nel cestino: da terminale la cancellazione è in genere **definitiva**.

Per guardare dentro un file non serve sempre un editor:

Comando	Descrizione
<code>cat</code>	scrive tutto il contenuto del file su stdout

<code>less</code>	per file lunghi: si scorre avanti e indietro e si cerca, senza riversare tutto sul terminale. q per uscire
<code>head / tail</code>	prime / ultime 10 righe. <code>head -n 5 dati.txt</code> = prime 5

Per lavorare su molti file insieme ci sono le **wildcard**, che espande la shell:

- `*` = **zero o più** caratteri qualsiasi
- `?` = **esattamente un** carattere

```
$ ls *.c
hello.c main.c prova.c
$ ls prova?.c
prova1.c prova2.c provaA.c
```



1.3 Flussi: redirectione e pipe

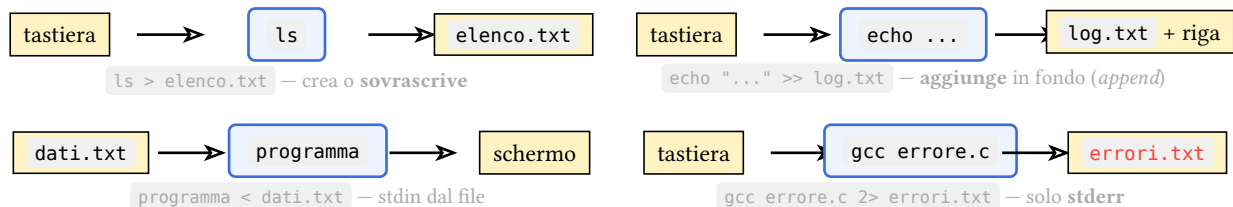
Ogni programma ha tre **flussi standard**:

n.	Flusso	Di solito
0	<code>stdin</code> — standard input	dalla tastiera
1	<code>stdout</code> — standard output	sul terminale
2	<code>stderr</code> — standard error	sul terminale



La shell può collegare ciascun flusso a qualcosa di diverso dal terminale.

Le **redirezioni** li collegano a un file invece che a tastiera e schermo:

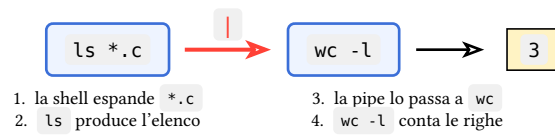


```
$ ls > elenco.txt           # sul terminale non compare niente
$ cat elenco.txt
hello.c
$ echo "prima riga" > log.txt
$ echo "seconda riga" >> log.txt
$ cat log.txt
prima riga
seconda riga
```

Con `<` il programma continua a leggere da `stdin`: è **la shell** che ha collegato `stdin` al file, il programma non se ne accorge.

`2>` redirige il file descriptor 2. Si separa `stderr` da `stdout` perché output normale ed errori hanno significati diversi e spesso si trattano in modo diverso.

La **pipe** `|` invece collega lo **stdout del primo** comando allo **stdin del secondo**.



Con un file temporaneo

```
$ ls > tmp.txt  
$ wc -l < tmp.txt
```

Con la pipe: niente file in mezzo

```
$ ls | wc -l  
$ ls | less # scorro l'elenco
```

Idea UNIX: piccoli programmi che si combinano per costruire operazioni più complesse.

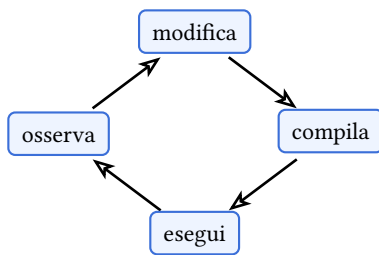
Micro-esercizio. Partendo dalla home: creare `lezione02`, entrarci, creare `a.c`, `b.c`, `note.txt` vuoti, salvare l'elenco dei soli `.c` in `sorgenti.txt`, aggiungerlo a `note.txt`, contare le righe di `note.txt`.

```
$ mkdir lezione02  
$ cd lezione02  
$ touch a.c b.c note.txt  
$ ls *.c > sorgenti.txt  
$ cat sorgenti.txt >> note.txt  
$ wc -l < note.txt  
2
```

2 Il linguaggio C

2.1 Dal sorgente all'eseguibile

Scrivere un programma C è un ciclo che si ripete:

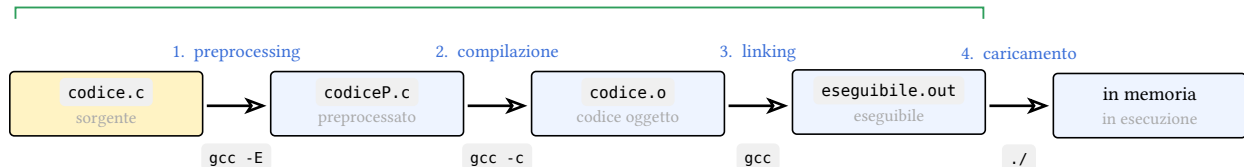


Il programma non nasce nell'editor: nasce da questo ciclo, ripetuto.

modifica ≠ **compilazione** ≠ **esecuzione**. Se cambi il sorgente e non ricompili, stai eseguendo il vecchio programma.

Il passo centrale è la compilazione. C è un linguaggio **compilato**: il sorgente `.c` è un semplice file di testo, e per diventare un programma che gira attraversa **quattro fasi**.

`gcc codice.c -o eseguibile.out` fa le fasi 1-3 con un solo comando



1. **Preprocessing**: vengono eseguite le **direttive per il pre-processore**, le righe che iniziano con `#`.
2. **Compilazione**: il sorgente viene tradotto in **codice oggetto** `.o`.
3. **Linking**: i file oggetto vengono collegati in un unico **eseguibile**.
4. **Caricamento**: quando lo lanci, l'eseguibile viene copiato in memoria e parte.

Da sapere — codice oggetto e linking (aggiunto, non spiegato in aula)

Il codice oggetto è il sorgente già tradotto in linguaggio macchina, ma non ancora eseguibile: gli mancano i pezzi che stanno altrove. Per esempio il codice di `printf` non è nel tuo file, sta nella libreria standard. Il **linker** mette insieme il tuo `.o` e questi pezzi, e ne esce un file che si può eseguire.

La prima fase lavora solo sul testo. Le direttive vengono eseguite **prima** della compilazione vera e propria: sostituiscono dei nomi simbolici con il contenuto che indicano. Due usi:

- `#include <stdio.h>` **include un file**: al suo posto viene incollato il contenuto di una libreria come `stdio.h`, o di un altro file come `my_lib.c`;
- `#define NOME testo` **definisce una macro**: da lì in poi ogni `NOME` viene sostituito con `testo`. Serve per costanti o comandi semplici.

Con `gcc -E` ci si ferma dopo il preprocessing e si vede cosa arriva al compilatore:

```
#include <stdio.h>
#define CONST_MESSAGE "Hello world\n"

int main(void) {
    printf(CONST_MESSAGE);
    return 0;
}
```

```
$ gcc -E codice.c -o codiceP.c
$ wc -l codiceP.c
845 codiceP.c    # 7 righe diventate 845
$ tail -4 codiceP.c
int main(void) {
    printf("Hello world\n");
    return 0;
}
```

Le righe in più sono il contenuto di `stdio.h`, incollato dall' `include`. In fondo c'è il nostro `main`, con `CONST_MESSAGE` già sostituita dal suo testo.

Di solito le fasi non si lanciano una per una: `gcc` (*GNU Compiler Collection*, il compilatore più diffuso) fa preprocessing, compilazione e linking con un solo comando.

```
$ gcc hello.c           # se va tutto bene non scrive niente
$ ls
a.out hello.c          # senza -o l'eseguibile si chiama a.out
$ gcc -Wall -Wextra -std=c11 hello.c -o hello # warning, C11, nome hello
$ ./hello
Hello, world!
$ echo $?
0
```

Pezzo	Significato
<code>-o hello</code>	dà il nome all'eseguibile
<code>-Wall -Wextra</code>	attiva i warning: non sono rumore , spesso segnalano un errore vero
<code>-pedantic</code>	segnala tutto ciò che non rispetta alla lettera lo standard del C
<code>-g</code>	aggiunge all'eseguibile le informazioni per il <i>debugger</i> (il programma che serve a cercare gli errori)
<code>-std=c11</code>	standard del C da usare (esistono C89, C95, C99, C11, C17, C23)
<code>./hello</code>	«esegui <code>hello</code> che sta nella directory corrente ». La shell cerca i comandi in un elenco di directory, e quella corrente non c'è per forza
<code>echo \$?</code>	stampa il valore restituito dall'ultimo programma. 0 = terminato correttamente

Le opzioni complete sono in `man gcc` e su gcc.gnu.org/onlinedocs/gcc. Volendo, le fasi si possono anche separare:

```
$ gcc -E codice.c -o codiceP.c # 1. solo preprocessing
$ gcc -c codice.c -o codice.o  # 1+2: si ferma al codice oggetto
$ gcc -c codiceP.c -o codice.o # oppure: compila il file già preprocessato
$ gcc codice.o -o eseguibile.out # 3. linking
$ ./eseguibile.out              # 4. caricamento ed esecuzione
Hello world
```

Nella quarta fase l'eseguibile viene caricato in memoria, divisa in quattro zone:

codice le istruzioni
dati statici variabili globali
heap
stack variabili locali

Ogni zona ha un compito. Il **codice** contiene le istruzioni del programma. I **dati statici** contengono le variabili globali, lo **stack** quelle locali: la differenza tra le due si vede nella prossima sezione.

Da sapere — stack e heap (aggiunto, non spiegato in aula)

Lo **stack** (pila) cresce e si svuota da solo: quando una funzione parte, le sue variabili locali vengono messe in cima; quando finisce, vengono tolte. L'**heap** è la memoria che il programma chiede esplicitamente mentre gira, e che resta finché non la restituisce.

Se qualcosa va storto, l'errore può arrivare in due momenti diversi:

```
int main(void) {
    puts("Hello, world!") // manca ;
    return 0;
}
```

```
hello.c:4:26: error: expected ';' before
'return'
```

Errore di compilazione	Errore di esecuzione
il programma non viene costruito	l'eseguibile esiste ma si comporta male o termina in modo inatteso

File, riga, messaggio: usali. Correggi **dall'alto verso il basso**: un solo errore può generare molti messaggi dopo.

3 errori banali che fanno perdere tempo

1. essere nella directory sbagliata;
2. modificare un file e compilarne un altro;
3. dimenticare di ricompilare.

Diagnosi minima

1. sono nella directory giusta?
2. il sorgente esiste?
3. ci sono errori o warning?
4. l'eseguibile è stato creato?
5. sto eseguendo quello giusto?

Routine utile: `pwd` → `ls` → `gcc ...` → `./programma`.

2.2 Com'è fatto un programma C

C segue il **paradigma imperativo**: il programma è una sequenza di istruzioni che, una dopo l'altra, cambiano i valori in memoria. Ed è **strutturato a blocchi**: il programma è una collezione di **funzioni**, e l'esecuzione parte sempre dalla funzione `main`. Un file C ha tre parti, in quest'ordine:

```
#include <stdio.h> // 1. direttive

int globalA; // 2. globali

int main(void) { // 3. funzioni
    puts("Hello!");
    return 0;
}
```

1. direttive per il pre-processore:

`#include <stdio.h>` rende disponibili le funzioni di input/output (come `puts`);

2. dichiarazioni globali: variabili visibili in tutto il programma;

3. funzioni: `main` prende niente (`void`) e restituisce un intero (`int`). `return 0` la termina e restituisce 0 al sistema operativo.

Le **graffe** `{ }` delimitano un blocco. Indentazione e nomi sensati non sono obbligatori, ma il programma deve essere leggibile. Per questo servono i **commenti**: testo racchiuso fra `/*` e `*/` (anche su più righe), oppure da `//` a fine riga. Il compilatore li ignora.

I commenti sono molto importanti: documenta **sempre** cosa fa un'istruzione, un blocco o una funzione, per ricordartelo quando rileggi.

Per stampare ci sono due funzioni, `puts` e `printf`:

Funzione	Cosa fa
<code>puts("Laboratorio 2");</code>	stampa una stringa e va a capo
<code>printf("Risultato: %d\n", 42);</code>	stringa formattata : <code>%d</code> viene sostituito dal valore. Non va a capo da sola : serve <code>\n</code>

`printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto dei *placeholder* `%d`, `%s`, ... Tutti i placeholder sono nella sezione sui tipi.

Mettendo insieme i pezzi:

```
#include <stdio.h>

int main(void) {
    int x = 21;    // una variabile

    puts("Laboratorio 2");
    puts("Lezione 1");
    printf("2 * %d = %d\n", x, 2 * x);

    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11
lezione01.c -o lezione01
$ ./lezione01
Laboratorio 2
Lezione 1
2 * 21 = 42
```

I nomi che scegli tu (di variabili, funzioni, tipi) sono **identificatori**: sequenze di lettere, cifre e `_` che iniziano con una lettera o con `_`. Le **parole chiave** del linguaggio (`int`, `return`, `while`, ...) sono riservate e non si possono usare come nomi.

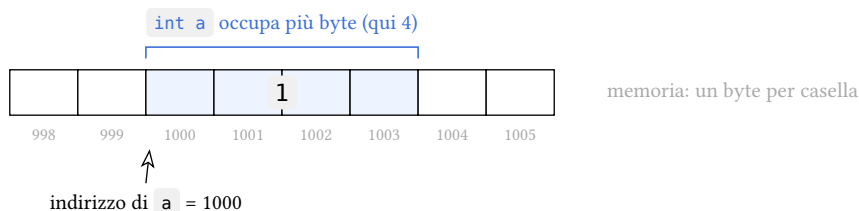
Nome	Valido	Perché
somma	sì	lettere
tmp	sì	inizia con <code></code>
x1	sì	la cifra non è all'inizio
1x	no	inizia con una cifra
mia-var	no	- non è ammesso
int	no	parola chiave

Nel programma di sopra `x` è una **variabile**: una **locazione di memoria** che contiene un **valore modificabile** di un certo **tipo**. L'insieme dei valori di tutte le variabili in un certo momento è lo **stato del programma**.

```
int a;           // dichiarazione
int a = 1;       // con valore iniziale
```

La **dichiarazione** dice al compilatore il tipo, e quindi **quanta memoria** riservare.

A ogni variabile è associato un **indirizzo**: un numero intero che indica dove si trova in memoria.



Una variabile però non è utilizzabile ovunque. Lo **scoping** è la regione del programma in cui una variabile è dichiarata, definita e utilizzabile: fuori da lì non è visibile. In C lo scoping è **statico**: viene deciso **a tempo di compilazione**, guardando dove sta la dichiarazione nel testo.

Globale: dichiarata fuori da ogni blocco e funzione. Sta nei **dati statici** ed è visibile nell'intero programma.

Locale: dichiarata dentro un blocco o una funzione. Sta sullo **stack** ed è visibile nel blocco e nei blocchi annidati dentro.

Una variabile locale in un blocco annidato può **coprire** una variabile con lo stesso nome, globale o di un blocco più esterno:

```
int g = 1; // globale

int main(void) {
    int x = 5; // locale a main
    {
        int g = 2; // copre la g globale
        printf("g = %d, x = %d\n", g, x);
    }
    printf("g = %d\n", g);
    return 0;
}
```

programma — vede `g = 1`

main — vede `g = 1` e `x`

blocco interno — vede `x` e la sua `g = 2`
la `g` globale qui è coperta

```
$ ./scope
g = 2, x = 5 # dentro il blocco
g = 1        # fuori
```

2.3 Tipi e costanti

C è **strongly typed**: ogni espressione e ogni variabile ha un tipo, noto al compilatore. I tipi di base sono quattro:

Tipo	Contiene
char	un intero grande un byte
int	numeri interi: 123, -45, 0; anche costanti carattere come 'a' o '1'
float	numeri reali
double	numeri reali in precisione doppia

Si possono modificare con quattro **qualifier**:

- `short` e `long` qualificano gli interi: `short int a;`. Da soli stanno per `short int` e `long int`. `long` può qualificare anche `double`;
- `signed` e `unsigned` qualificano `char` e `int`: `unsigned` = solo valori ≥ 0 .

Quanti byte occupa ogni tipo **dipende dalla macchina**: il C garantisce solo `short ≤ int ≤ long`. Per saperlo si usa l'operatore `sizeof`, che restituisce la dimensione in byte:

```
int main(void) {
    int a = sizeof(int);
    printf("%d\n", a); // stampa 4
    return 0;
}
```

char		1 byte
short		2 byte
int		4 byte
long		8 byte
float		4 byte
double		8 byte

Misurate così su questo computer (Linux a 64 bit):

`sizeof` si applica anche alle **espressioni**, e lì c'è una sorpresa (`%zu` è il placeholder di `printf` per il risultato di `sizeof`):

Da sapere — ottale ed esadecimale (aggiunto, non spiegato in aula)

Sono lo stesso numero scritto in base 8 o in base 16 invece che in base 10. In esadecimale le cifre dopo il 9 sono A =10, B =11, ..., F =15.

- $014 = 1 \cdot 8 + 4 = 12$
- $0x3A = 3 \cdot 16 + 10 = 58$

Attenzione: lo 0 iniziale non è decorativo. 010 vale 8, non 10.

Il carattere `\` è un carattere speciale di **escape**: insieme al carattere dopo indica un carattere che non si può scrivere direttamente, perché non stampabile o già usato dalla sintassi.

'\n'	'\t'	'\0'	'\\'
a capo	tabulazione	carattere nullo (codice 0)	la barra \ stessa

Le costanti stringa sono memorizzate come **array di caratteri**, cioè caratteri uno dopo l'altro in memoria:

Da sapere — il terminatore '\0' (aggiunto, non spiegato in aula)

In fondo a ogni stringa il compilatore aggiunge `'\0'`, che segna dove finisce. Per questo `sizeof("ciao!\n")` vale 7: sei caratteri più il terminatore.

c	i	a	o	!	\n	\0
---	---	---	---	---	----	----

Ora che i tipi sono chiari si può completare `printf`: ogni tipo ha il suo placeholder, e **il placeholder deve essere compatibile con il tipo del valore**. Ognuno è fatto così:



`printf("%08.2f", 3.14);` → `00003.14` — forma generale: `%[flags][width][.precision][length]specifier`

Interi		Altri tipi	
<code>%d</code> , <code>%i</code>	int con segno	<code>%f</code>	double
<code>%u</code>	unsigned int	<code>%e</code> , <code>%E</code>	notazione scientifica
<code>%x</code> , <code>%X</code>	esadecimale	<code>%g</code> , <code>%G</code>	formato compatto
<code>%o</code>	ottale	<code>%c</code>	carattere
<code>%ld</code>	long	<code>%s</code>	stringa
<code>%lld</code>	long long (ancora più lungo)	<code>%p</code>	indirizzo di memoria
<code>%zu</code>	<code>size_t</code> : il risultato di <code>sizeof</code>	<code>%%</code>	il carattere %

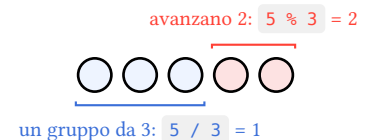
2.4 Operatori ed espressioni

Gli operatori combinano valori e variabili in **espressioni**. Sono di quattro famiglie:

Famiglia	Operatori
aritmetici	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>
logici e di confronto	<code>!</code> , <code>&&</code> , <code> </code> fra valori booleani; <code>==</code> , <code>!=</code> , <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> fra valori numerici
di assegnamento	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code>
incremento e decremento	<code>++</code> , <code>--</code>

Gli **aritmetici** sono addizione, sottrazione, moltiplicazione, divisione e modulo (il resto della divisione). Il risultato dipende dal tipo degli operandi: tra due interi la divisione è **intera**, la parte decimale si butta.

```
5 / 3      // 1      divisione intera
5.0 / 3.0  // 1.66667 divisione tra reali
5 % 3      // 2      resto
```



Il risultato deve stare nel tipo dell'espressione: se è troppo grande (o troppo piccolo) per i byte a disposizione si ha **overflow** (o **underflow**), e il valore che esce è sbagliato.

Gli operatori **logici** sono tutti binari tranne `!`. In C **non esiste un tipo booleano primitivo tradizionale**: si usano i numeri. **0 è falso, qualsiasi valore diverso da 0 è vero**. Il risultato di un operatore logico o di confronto è sempre **0 se falso, 1 se vero**:

a	b	!a	a && b	a b
0	0	1	0	0
0	-3	1	0	1
5	0	0	0	1
5	-3	0	1	1

`&&` e `||` hanno una **valutazione cortocircuitata**: la valutazione si ferma appena il risultato è noto. Se in `a && b` la `a` vale 0, il risultato è già 0 e `b` non viene nemmeno calcolata; se in `a || b` la `a` è diversa da 0, il risultato è già 1. Così si ottiene un risultato anche quando la parte dopo darebbe errore:



Attenzione a scrivere i confronti come in matematica. `2 < a < 5` **non** controlla che `a` sia tra 2 e 5: equivale a `(2 < a) < 5`, e `2 < a` vale 0 o 1, che è sempre minore di 5. Con `a = 7`:

```
a = 7      2 < a < 5
prima 2 < a      1 < 5
risultato      1 (vero!)
```

La forma corretta è `2 < a && a < 5`, che con `a = 7` dà 0. Con `-Wall` il compilatore se ne accorge:

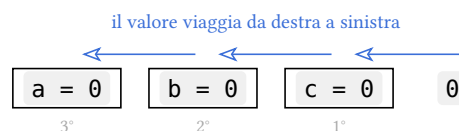
```
warning: comparisons like 'X<=Y<=Z' do not have their
mathematical meaning [-Wparentheses]
```

L'**assegnamento** ha la forma `variabile = parte_destra`. La parte destra è un'espressione, che può essere a sua volta un assegnamento: l'assegnamento stesso restituisce il valore assegnato. Per questo si può scrivere l'**assegnamento multiplo**, che è **associativo da destra**: si esegue partendo dall'ultimo.

```

a = b = c = 0; // a = (b = (c = 0))
a += 5;        // a = a + 5
a = 3 + 4 * 2; // 11: prima il calcolo

```



Gli operatori di assegnamento hanno **priorità più bassa** degli altri: nell'ultima riga prima si calcola tutta la parte destra, poi si assegna.

Attenzione: i tipi devono essere **compatibili**, sia dentro l'espressione sia tra l'espressione e la variabile a cui viene assegnato il valore.

++ e **--** aumentano o diminuiscono di 1. La posizione conta quando il valore viene usato:

Con a = 5	Cosa fa	Risultato
x = ++a;	prima incrementa, poi restituisce il nuovo valore	x = 6 , a = 6
x = a++;	restituisce il valore attuale , poi incrementa	x = 5 , a = 6

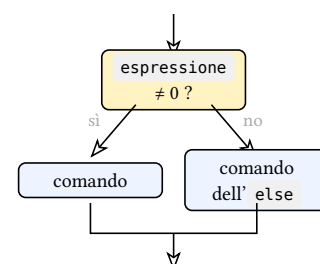
2.5 Il controllo del flusso

Le istruzioni di un blocco vengono eseguite in ordine. I **comandi** condizionali e i cicli cambiano quest'ordine, e decidono guardando un'espressione: come per gli operatori logici, **diversa da 0 = vera**, **0 = falsa**.

```

if (espressione)
    comando;
else
    comando;

```



Se l'espressione è diversa da 0 si esegue il primo comando, altrimenti quello dell' **else**. L' **else** si può omettere, e gli **if** si possono annidare uno dentro l'altro.

I **cicli** ripetono un blocco di codice finché l'espressione resta diversa da 0; quando diventa 0 si prosegue dopo il ciclo. Il C ne ha tre:

```

while (espressione)
    comando;

```

```

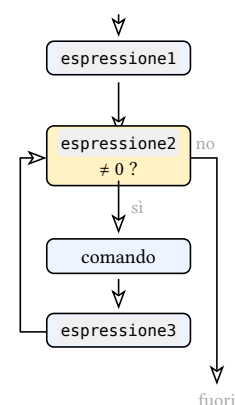
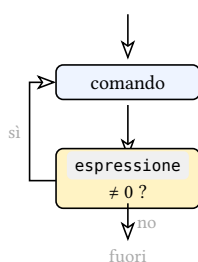
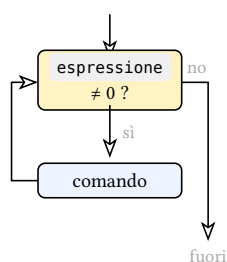
do
    comando;
while (espressione);

```

```

for (espressione1;
     espressione2;
     espressione3)
    comando;

```



Il `while` controlla l'espressione **prima** di ogni giro: se è falsa subito, il corpo non viene mai eseguito. Il `do-while` la controlla **dopo**: il corpo viene eseguito **almeno una volta**. Esempio: stampare le cifre di un numero, dall'ultima alla prima. `n % 10` è l'ultima cifra, `n /= 10` la toglie.

```
int n = 12345;
while (n > 0) {
    printf("%d\n", n % 10);
    n /= 10;
}
```

n	stampa n % 10	poi n diventa
12345	5	1234
1234	4	123
123	3	12
12	2	1
1	1	0
0	n > 0 falso: fine	

Nel `for`, `espressione1` viene valutata **una sola volta**, prima del primo giro. Poi, finché `espressione2` è vera, si esegue il corpo, e alla fine di ogni giro si valuta `espressione3`. È quindi solo un modo compatto di scrivere un `while`. Ognuna delle tre espressioni si può omettere, come la prima qui a destra:

```
for (e1; e2; e3)
    comando;
```

```
e1;
while (e2) {
    comando;
    e3;
}
```

```
int n = 12345;
for (; n > 0; n /= 10)
    printf("%d\n", n % 10);
```

Attenzione: un ciclo può **non terminare mai**, se l'espressione non diventa mai 0.

Due istruzioni cambiano il giro dall'interno:

- `break` causa l'**uscita immediata** dal ciclo (o dallo `switch`) che la contiene;
- `continue` **termina il giro corrente** e passa al successivo. Può comparire solo in `for`, `while` e `do-while`.

```
int a = 0;
while (1) {                // sempre vero
    if (a == 10) break;
    a++;
}                          // qui a vale 10
```

```
for (int k = 0; k < 5; k++) {
    if (k == 2) continue;
    printf("%d ", k);
}                          // 0 1 3 4
```

Il ciclo `while (1)` non finirebbe mai: è il `break` che lo chiude.

Con `k = 2` il `printf` viene saltato, ma il ciclo va avanti.

Infine lo `switch`, un'istruzione condizionale a **scelta multipla**. L'espressione deve essere di tipo **intero**; ogni `case` ha un'etichetta costante, e le etichette devono essere **uniche**. Il controllo salta all'etichetta uguale al valore dell'espressione, o a `default` se nessuna corrisponde.

```

switch (espressione) {
    case espr_costante:
        comando;
        break;
    case espr_costante:
        comando;
        break;
    default:
        comando;
}

```

Il `break` qui è essenziale: **senza `break` vengono eseguite anche le istruzioni dei `case` successivi**. Con `c = 2`:

```

case 1: puts("uno");
case 2: puts("due"); ← entra qui; niente break : prosegue ↓
case 3: puts("tre"); break; → esce
default: puts("altro");

```

Stampa `due` e poi `tre`. Con `-Wextra` gcc avvisa: `this statement may fall through`.